Science-Aware Retrieval Operators for AI-Driven HPC Data Discovery

Author A and Author B
Department of Computer Science
University Name, City, Country
{authora, authorb}@university.edu

***Abstract*—AI agents are transforming scientific computing — orchestrating experiments across national facilities, writing research papers, and querying workflow provenance via MCP. Yet agents consistently fail at data retrieval: ScienceAgentBench (ICLR 2025) shows the best agent achieves only 32% on scientific tasks, with failures traced to data handling rather than reasoning. The root cause: retrieval systems treat scientific content as plain text. Agentic RAG systems improve search orchestration but remain domain-agnostic — none incorporate science-aware numerical retrieval operators. While recent systems such as PANGAEA-GPT search geoscientific archives, none perform arithmetic dimensional conversion at retrieval time. Embedding-based approaches achieve only 0.54 accuracy on numerical content, barely above chance.**

We present clio-agentic-search, a retrieval engine that introduces pluggable science-aware operators as first-class retrieval branches alongside standard BM25 and dense vector search. A dimensional-analysis operator canonicalizes physical quantities to base SI units via explicit arithmetic conversion (kPa $\times$ 10^3 = Pa), guaranteeing cross-prefix matching by construction — unlike string normalization or learned embeddings. A formula-normalization operator matches mathematical expressions across notation variants. These operators execute as parallel branches within a federated multi-namespace architecture that queries filesystems, object stores, HDF5 files, NetCDF datasets, and vector databases through per-backend capability negotiation. An agentic loop with LLM-driven query rewriting enables iterative multi-hop search with automatic SI unit expansion over federated scientific data.

We evaluate on scientific retrieval tasks spanning five measurement domains, demonstrating that science-aware operators recover results missed by all domain-agnostic baselines.

I. Introduction

Scientific computing generates data at unprecedented scale. A single experimental campaign at a DOE facility may produce terabytes of simulation outputs in HDF5, observational records on parallel filesystems, derived features in object stores, and annotations in knowledge graphs. When a scientist later asks, "which experiments measured pressures between 190 and 360 kPa?"—a question that should be trivial—no existing retrieval system can answer it reliably. The measurements exist, scattered across storage tiers in different unit representations: "200 kPa" on one filesystem, "200000 Pa" in an HDF5 attribute, "0.2 MPa" in an S3-hosted dataset. These describe identical pressure. Every retrieval system treats them as unrelated strings.

This data discovery crisis is intensifying as AI agents enter scientific workflows. Coscientist [1] autonomously plans and executes wet-lab synthesis. AI Scientist v2 [2] generates and writes up research end-to-end. Autonomous agents at ORNL orchestrate experiments across national facilities [3], and the Model Context Protocol now wraps Globus search, compute, and storage under a unified agent interface for HPC [4]. The trajectory is clear: agents are transitioning from processing scientific literature to operating scientific infrastructure. Yet these agents consistently fail at data retrieval, not reasoning. ScienceAgentBench [5], a benchmark of 44 tasks drawn from peer-reviewed publications, reports that the best agent achieves only 32.4% end-to-end success, with failures traced to data handling rather than reasoning. MLAgentBench [6] observes the same pattern. A comprehensive survey of agentic AI [7] identifies data management as explicitly underexplored. These agents can reason about science—they cannot find the data they need to reason about.

The root cause is structural: retrieval systems treat scientific content as plain text. Three compounding failures explain why.

Dimensional blindness. Scientific measurements are routinely expressed in different unit prefixes. String-based quantity normalization [8] maps unit names to canonical strings, but "kilopascal" and "pascal" remain different strings—no string operation bridges the SI prefix gap. Learned embeddings such as CONE [9] encode numbers and units in a shared vector space, but the Numeracy Gap study [10] evaluates 13 embedding models and finds 0.54 binary retrieval accuracy on numerical content—effectively random. NC-Retriever [11] reports 16.3% dense retrieval accuracy on numeric constraint queries. The fundamental problem is that none of these approaches performs the required arithmetic: $200 \times 1000 = 200{,}000$.

Formula opacity. "$F = ma$," "$F = m \cdot a$," and "$ma = F$" encode the same physical law but produce distinct token sequences. Math-aware search systems [12], [13] match equation structure in LaTeX markup but cannot connect a matched formula to the physical measurements it governs. No retrieval system combines formula normalization with quantity-aware dimensional search.

Storage fragmentation. HPC data spans parallel filesystems, S3 object stores, vector databases, and binary formats such as HDF5 and NetCDF. No system provides unified search with science-aware operators across these backends. RAGRoute [14] federates generic text search. PROV-IO+ [15] captures HDF5 provenance with under 3.5% overhead but exposes no search interface.

These failures compound. The motivating query—"find experiments where pressure was between 190–360 kPa across all storage"—requires dimensional conversion across SI prefixes, federated search across heterogeneous backends, and potentially formula context linking pressure to related variables. No existing system addresses even two of these three requirements simultaneously.

We hypothesize that retrieval systems can overcome these failures through *science-aware operators*: deterministic, pluggable retrieval primitives that encode domain knowledge as explicit arithmetic and symbolic transformations, executing as first-class branches alongside standard BM25 and dense vector search. Unlike string normalization or learned embeddings, these operators make correctness verifiable by inspection—if the SI conversion factor is correct, the match is correct. We test this hypothesis through five contributions:

1) **Dimensional-conversion retrieval.** We canonicalize physical quantities to base SI units via arithmetic multiplication (e.g., kPa $\times$ 10^3 = Pa) across 13 units in 5 domains, with guaranteed correctness by construction.
2) **Formula normalization.** We normalize mathematical expressions across whitespace, superscript, side-swap, and factor-reordering variants, integrated with dimensional operators as parallel retrieval branches.
3) **Federated multi-namespace search.** A connector architecture supports unified retrieval across filesystem, S3, Qdrant, Neo4j, HDF5, and NetCDF backends with per-backend capability negotiation and incremental indexing.
4) **HDF5/NetCDF metadata indexing.** Dedicated connectors extract measurements from scientific file formats and feed them through the same SI canonicalization pipeline, enabling cross-format dimensional search.
5) **Multi-hop agentic retrieval.** An LLM-driven query rewriting loop iteratively refines searches using expand, narrow, and pivot strategies, converging within 2–3 hops.

The remainder of this paper is organized as follows. Section 2 surveys hybrid retrieval, the numeracy crisis, quantity-aware retrieval, and scientific data discovery. Section 3 presents the system design. Section 4 describes the implementation. Section 5 evaluates across dimensional conversion, formula matching, federated search, and ablation. Section 6 concludes.

II. Background and Related Work

The gap between what retrieval systems deliver and what scientific data discovery demands spans four areas: the hybrid retrieval foundations we build upon, the numeracy crisis that motivates explicit arithmetic operators, quantity-aware retrieval systems that attempt but fail to solve cross-unit matching, and scientific data discovery in HPC.

A. Hybrid Retrieval Foundations

Modern retrieval combines lexical and dense signals. BM25 [16] remains the strongest lexical baseline; dense retrieval via learned bi-encoders [17] captures semantic similarity that keyword matching misses. Fusing these signals is now standard practice. Bruch et al. [18] show that a learned convex combination consistently outperforms reciprocal rank fusion (RRF) on TREC benchmarks, establishing the analytic framework we adopt. Learned sparse methods—SPLADE [19] and its Mistral-SPLADE variant—bridge the gap by producing sparse lexical-weight vectors from dense encoders. BGE-M3 [20] unifies dense, sparse, and colBERT-style multi-vector retrieval in a single model trained on multilingual data, demonstrating that multi-granularity fusion improves recall across diverse query types. At the agentic frontier, A-RAG [21] introduces hierarchical retrieval interfaces that let LLM agents compose retrieval strategies dynamically.

We build *on* hybrid retrieval, not beside it. Our science-aware operators execute as parallel branches alongside BM25 and dense search within the same fusion pipeline, adding domain-specific recall without replacing any existing component.

B. The Numerical Reasoning Crisis

Numbers are fundamentally broken in both retrieval systems and large language models. The Numeracy Gap study [10] evaluates 13 embedding models on numerical content and reports 0.54 binary retrieval accuracy—barely above random chance. NC-Retriever [11] finds that dense retrieval achieves only 16.3% accuracy on queries with numeric constraints, even when the constraint is stated explicitly in both query and document. The root cause is architectural: tokenizers encode numbers per-digit rather than by magnitude, so "200" and "200000" share no meaningful representation [22]. NumericBench [23] confirms these failures across arithmetic, comparison, and unit conversion tasks in LLMs. The Number Cookbook [24] systematically catalogs failure modes. GSM-Symbolic [25] delivers the sharpest result: changing only the numeric values in grade-school math problems while preserving problem structure causes a 65% accuracy drop, proving that LLMs pattern-match on surface numerals rather than reason about magnitude.

This is not merely a unit problem. Numbers themselves—the substrate of all scientific measurement—are broken in every retrieval and language model. Scientific data retrieval, where matching requires comparing magnitudes across units, is especially vulnerable.

C. Quantity-Aware Retrieval

Three lineages attempt to make retrieval quantity-aware. None succeeds at arithmetic conversion.

String normalization. QFinder [26] extends Elasticsearch with quantity-aware ranking by extracting and normalizing numeric expressions. CQE [27] advances this with dependency parsing over a 531-unit dictionary, handling complex quantity expressions. Numbers Matter! [8] builds a full retrieval pipeline with quantity-aware indexing and evaluation on FinQuant and MedQuant benchmarks. The fundamental

limitation persists: string normalization cannot cross SI prefixes. "kilopascal" and "pascal" remain distinct strings after normalization, so a query for "200 kPa" never matches a document containing "200000 Pa."

Learned embeddings. CONE [9] trains a transformer to embed number–unit pairs into a shared vector space, achieving a 25% Recall@10 gain on quantity-bearing queries. NC-Retriever [11] applies contrastive learning to numeric constraint matching. These approaches are probabilistic: a model may learn that 200 kPa is *similar* to 200000 Pa, but provides no correctness guarantee. Weller et al. [28] prove theoretical upper bounds on embedding-based retrieval, showing that fixed-dimensional embeddings cannot faithfully preserve all distance relationships—a result that undercuts learned approaches to exact quantity matching.

Math-aware search. Approach0 [12] matches equation structure via operator trees. SSEmb [13] learns semantic similarity over mathematical expressions. MIRB [29] provides the first unified math information retrieval benchmark across 12 datasets. These systems match formula *structure* but are disconnected from physical measurements—they cannot link $F = ma$ to a document reporting force in newtons.

Gap. No existing retrieval system performs arithmetic SI conversion to canonicalize measurements. None combines quantity matching with formula normalization. None operates across federated HPC backends.

D. Scientific Data Discovery in HPC

Paper search. OpenScholar [30] indexes 45 million open-access papers and achieves GPT-4o-level citation quality. HiPerRAG [31] scales RAG to 3.6 million articles across DOE supercomputers Polaris, Sunspot, and Frontier. Both search *papers*—not experimental data, simulation outputs, or HDF5 files.

Data search. Recent systems do search scientific data. PANGAEA-GPT [32] deploys hierarchical multi-agent search over 400,000+ geoscientific datasets with data-type-aware routing and unit scale validation, achieving 8.14/10 relevance versus a 2.87/10 baseline. LLM-Find [33] uses LLM-generated code to retrieve geospatial data from heterogeneous sources with 80–90% success. The distinction is temporal: PANGAEA-GPT validates unit scales *post-hoc* (after retrieval); we convert *at retrieval time* (during indexing and query processing). LLM-Find generates access code per source; we provide reusable retrieval operators.

HPC data management. Datum [34] catalogs scientific data at INL but offers no content search. BRINDEXER [35] indexes POSIX metadata on Lustre with 69% query improvement—metadata only, no content. PROV-IO+ [15] captures HDF5 I/O provenance with under 3.5% overhead but exposes no search interface. HDF Clinic [36] proposes a knowledge graph over HDF5 but remains unimplemented.

Agentic retrieval. The SoK on Agentic RAG [37] provides the first systematization of knowledge in this space. Context-1 [38] demonstrates 20-billion-token agentic search. CLADD [39] augments collaborative drug-discovery agents with domain knowledge graph operators. SciAgent [40] coordinates specialist worker agents under a hierarchical controller. ScienceClaw [41] deploys a 300+ skill agent swarm with immutable artifact provenance. All improve orchestration. None incorporates science-aware numerical operators—arithmetic dimensional conversion or formula normalization—into the retrieval pipeline itself.

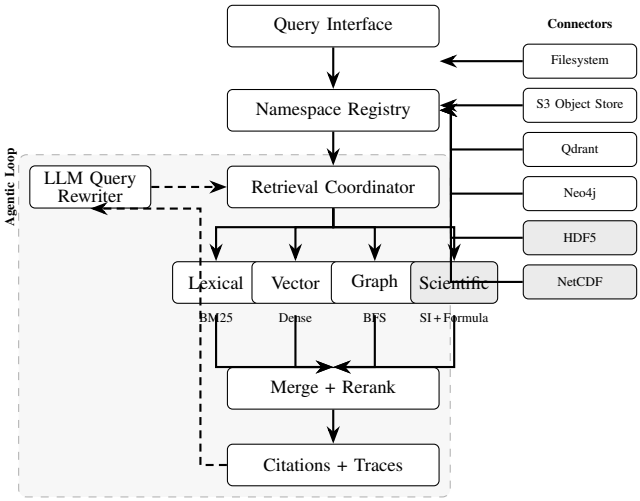

Fig. 1. System architecture of CLIO-AGENTIC-SEARCH. Science-aware components (shaded) execute alongside standard retrieval branches. The agentic loop iteratively refines queries via LLM rewriting.

Table I summarizes the landscape. No prior system supports any of our first five capabilities: arithmetic SI conversion, formula normalization, science-aware retrieval branches, federated multi-backend search, or HDF5/NetCDF indexing. We honestly mark capabilities we lack—we do not index papers. PANGAEA-GPT receives partial credit for quantity extraction (unit scale validation, but post-hoc rather than at retrieval time).

III. System Design

To address the three failures identified in Section 1, we design clio-agentic-search around a central principle: science-aware operators must execute as first-class retrieval branches, not post-processing filters. The architecture comprises a staged pipeline with four parallel branches (Sec. 3.1), dimensional conversion operators that perform explicit SI arithmetic (Sec. 3.2), deterministic formula normalization (Sec. 3.3), a hybrid fusion strategy (Sec. 3.4), federated multi-namespace search (Sec. 3.5), and an LLM-driven multi-hop retrieval loop (Sec. 3.6).

A. Architecture Overview

The system is a staged pipeline with four parallel retrieval branches (Fig. 1). A query enters through the Query Interface (FastAPI REST endpoint or CLI), which parses the raw query string, extracts typed scientific constraints (numeric ranges, unit filters, formula patterns), and forwards a structured query object to the Namespace Registry. The registry resolves which

TABLE I

CAPABILITY COMPARISON OF RETRIEVAL AND SCIENTIFIC DATA SYSTEMS. COLUMNS 1–5 CORRESPOND TO OUR CONTRIBUTIONS. ✓ = SUPPORTED, ✗ = NOT SUPPORTED, ○ = PARTIAL.

	SI conversion	*Formula norm.*	*Science branches*	*Federated backends*	*HDF5/NetCDF index*	*Agentic multi-hop*	*Quantity extraction*	*Paper search*	*Data search*	*LLM integration*
Ours	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓
PANGAEA-GPT	✗	✗	✗	✗	✗	✓	○	✗	✓	✓
OpenScholar	✗	✗	✗	✗	✗	✗	✗	✓	✗	✓
HiPerRAG	✗	✗	✗	✗	✗	✗	✗	✓	✗	✓
Numbers Matter!	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗
CONE	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗
CLADD	✗	✗	✗	✗	✗	✓	✗	✗	✗	✓
SciAgent	✗	✗	✗	✗	✗	✓	✗	✗	✗	✓
Context-1	✗	✗	✗	✗	✗	✓	✗	✗	✗	✓

namespaces—each backed by one of six connector types—are eligible for the query based on capability negotiation (Sec. 3.5).

The Retrieval Coordinator fans the query out to four parallel branches: lexical (BM25), vector (dense embedding similarity), graph (entity-relationship traversal), and scientific (dimensional measurement and formula queries). Each branch executes independently and returns scored candidate chunks. The Merge and Rerank stage deduplicates candidates by chunk identifier, applies hard scientific filters, computes a composite score via heuristic fusion, and emits the final top-K results with source citations.

Six connectors mediate access to heterogeneous backends: local filesystem, S3-compatible object stores, Qdrant vector database, Neo4j graph database, HDF5 (via h5py), and NetCDF (via xarray). Each connector declares a capability set specifying which retrieval branches it supports, enabling the coordinator to skip unsupported branches rather than fail. Every pipeline stage emits structured trace events for observability.

B. Dimensional Conversion Operators

The core insight behind dimensional conversion is that cross-unit matching is an arithmetic problem, not a similarity problem. When a scientist queries for pressures between 190 and 360 kPa, the system must recognize that a document reporting "250000 Pa" falls within that range. This requires multiplying 190 by 1000 to obtain 190000 Pa and 360 by 1000 to obtain 360000 Pa—a computation that no string normalization or learned embedding can guarantee.

We define a conversion table covering five SI domains and 13 units (Table II). Each entry maps a source unit to a canonical base unit and a multiplicative conversion factor. At index time, a regex-based measurement extractor identifies numeric-unit pairs in document text and scientific file metadata. Each extracted measurement is stored as a 4-tuple: (raw_value, raw_unit, canonical_value, canonical_unit). For example, "200 kPa" yields (200.0, "kPa", 200000.0, "Pa"). These tuples populate a `scientific_measurements` table in DuckDB, indexed on canonical_value and canonical_unit for efficient range scans.

TABLE II

SI CONVERSION TABLE. 5 DOMAINS, 13 UNITS. EACH SOURCE UNIT MAPS TO A CANONICAL BASE UNIT VIA A DETERMINISTIC MULTIPLICATIVE FACTOR.

Domain	**Source units**	**Canonical**	**Factors**
Distance	mm, cm, m, km	m	10^{-3}, 10^{-2}, 1, 10^{3}
Mass	mg, g, kg	kg	10^{-6}, 10^{-3}, 1
Time	s, min, h	s	1, 60, 3600
Pressure	Pa, kPa, MPa	Pa	1, 10^{3}, 10^{6}
Velocity	m/s, km/h	m/s	1, $\frac{1}{3.6}$

At query time, three operators execute against the canonicalized measurement store. The **numeric-range** operator parses range expressions of the form `low:high:unit`, converts both bounds to canonical units, and issues a DuckDB range predicate: a query "190:360:kPa" becomes a scan for canonical_value between 190000 and 360000 with canonical_unit = "Pa". The **unit-match** operator filters chunks containing any measurement in a specified unit domain regardless of value. The **value-match** operator retrieves measurements matching a specific canonical value within a configurable tolerance (default $\pm 1\%$).

The correctness guarantee distinguishes this approach from all prior work. Consider matching "200 kPa" against "200000 Pa". Arithmetic conversion computes $200 \times 1000 = 200000$ and succeeds by exact equality. String normalization maps "kilopascal" to one string and "pascal" to another—they never match. Learned embeddings [9] encode the two quantities as vectors in a shared space, but the Numeracy Gap study [10] reports 0.54 mean binary retrieval accuracy across 13

embedding models on numerical content, effectively random against a 0.50 baseline. Weller et al. [28] prove theoretical upper bounds showing that fixed-dimensional embeddings cannot faithfully preserve all distance relationships, a result that fundamentally undercuts learned approaches to exact quantity matching. Our operators are correct by construction: if the SI conversion factor is correct, the match is correct.

C. Formula Normalization

Scientific formulas appear in inconsistent surface forms across documents. "$F = m*a$", "$F = ma$", "$F = m \cdot a$", and "$ma = F$" express the same physical law but produce distinct token sequences that defeat both keyword and embedding retrieval. We normalize formulas through a deterministic six-step pipeline.

First, all whitespace is stripped. Second, all alphabetic characters are lowercased. Third, superscript notations are unified: LaTeX-style `^{N}` and Python-style `**N` are both normalized to `^N`. Fourth, the expression is split on the equality sign into left-hand and right-hand sides. Fifth, within each side, multiplicative factors are sorted lexicographically, so that `m*a` and `a*m` produce the same canonical form. Sixth, the two sides are rejoined with a normalized equality separator. The canonical form of all four variants above is `a*m=f`.

Normalized formulas are stored in a `scientific_formulas` table in DuckDB alongside their source chunk identifiers and the raw original text. At query time, the user's formula expression passes through the same six-step pipeline before lookup. Formula matching integrates with the dimensional conversion operators as a parallel branch in the scientific retrieval stage: a query can simultaneously request formulas containing a specific equation and measurements within a numeric range, and both constraints execute as independent DuckDB queries whose results are intersected at the merge stage.

D. Hybrid Retrieval Pipeline

The retrieval pipeline executes four branches in parallel and merges their results through a deterministic fusion strategy (Fig. 3).

Lexical branch. BM25 with parameters $k_1 = 1.2$ and $b = 0.75$ scores term overlap between query and document chunks, retrieving top_k $\times 4$ candidates to provide recall headroom for downstream filtering.

Vector branch. Chunks are embedded with MiniLM-L6-v2 (384-dimensional vectors, cosine similarity) and indexed in Qdrant. This branch retrieves top_k $\times 4$ candidates.

Graph branch. For Neo4j-backed namespaces, breadth-first traversal at depth 1 from query entities retrieves related chunks through typed relationships (e.g., cites, measures, uses_formula). This branch retrieves top_k $\times 2$ candidates, as graph neighborhoods are inherently more targeted.

Scientific branch. The dimensional conversion and formula normalization operators (Secs. 3.2–3.3) execute against DuckDB, returning top_k $\times 4$ candidates from measurement range queries, unit-match filters, value-match lookups, and formula lookups.

Merge and rerank. Candidates from all branches are deduplicated by chunk_id, retaining the maximum per-branch score. A scientific hard filter then applies: if the query specifies a numeric range or unit constraint, candidates not returned by the scientific branch are discarded—dimensional correctness is enforced, not merely preferred. A metadata filter removes candidates violating explicit metadata constraints (e.g., date range, file type). Surviving candidates are scored by a heuristic weighted sum:

$$\text{score} = w_l \cdot s_{\text{lexical}} + w_v \cdot s_{\text{vector}} + w_m \cdot s_{\text{metadata}} + w_s \cdot s_{\text{scientific}}$$

where the weights w_l, w_v, w_m, w_s are configurable per deployment. The top-K candidates by composite score constitute the final result set. Each result carries provenance: the source namespace, chunk identifier, file path, and the set of branches that contributed to its retrieval.

E. Federated Multi-Namespace Search

Scientific data at HPC facilities spans heterogeneous backends. A single experimental campaign may deposit raw output on a parallel filesystem, preprocessed arrays in HDF5, derived features in a vector database, and annotations in a knowledge graph. Rather than require data migration, we bring the retrieval operators to the data.

Every storage backend is wrapped by a connector that implements the `NamespaceConnector` protocol and declares a `RetrievalCapabilities` object specifying which retrieval branches it supports. The capability matrix is as follows:

- **Filesystem**: lexical, vector, scientific
- **S3**: lexical, vector, scientific
- **Qdrant**: vector
- **Neo4j**: graph
- **HDF5**: metadata, scientific
- **NetCDF**: metadata, scientific

The Retrieval Coordinator consults this matrix before dispatching. When a query spans multiple namespaces, the coordinator executes the single-namespace pipeline independently for each namespace, activating only the branches that the namespace's connector supports. Results from all namespaces are collected into a global candidate pool, sorted by composite score, deduplicated by chunk_id (since the same document may appear in multiple backends), and truncated to the final top-K.

Incremental indexing avoids redundant reprocessing. Each connector tracks indexed documents by SHA-256 content hash and modification time. On re-index, only documents whose hash or mtime has changed are reprocessed, enabling efficient index maintenance over evolving corpora without full rebuild.

F. Multi-hop Agentic Retrieval

Single-pass retrieval often fails for complex scientific questions that require iterative refinement. The multi-hop agentic retrieval loop automates this process.

The loop proceeds as follows. The initial query executes the full hybrid pipeline described in Section 3.4 across all federated namespaces. The retrieved results, along with the original query and the retrieval trace, are passed to an LLM (Anthropic API) that evaluates result quality and selects one of four strategies: **expand** (broaden the query to increase recall), **narrow** (add constraints to increase precision), **pivot** (reformulate the query to pursue a different aspect), or **done** (terminate the loop). If the strategy is not done, the LLM generates a rewritten query, which re-enters the pipeline for another full retrieval pass. Results from all hops are merged, deduplicated, and re-ranked. The loop terminates when the LLM selects done, when results stabilize across consecutive hops (no new unique chunk_ids appear), or when a configurable maximum hop count is reached.

Each hop constitutes a full pipeline execution: all four retrieval branches, all eligible namespaces, merge and rerank. This ensures that every rewritten query benefits from the same dimensional conversion, formula normalization, and federated search capabilities as the original. The LLM's role is strictly query rewriting and termination decision—it does not score or filter results, preserving the deterministic correctness guarantees of the science-aware operators. When the Anthropic API is unavailable, the system falls back to an offline strategy that expands queries with SI unit variants (e.g., appending "Pa" and "MPa" to a query containing "kPa"), providing degraded but functional multi-pass retrieval without LLM dependency.

IV. Implementation

clio-agentic-search is implemented as a single-binary Python system with no external service dependencies beyond an optional LLM endpoint. We cover the software stack and storage schema (Sec. 4.1), connectors for binary scientific formats (Sec. 4.2), and the experimental platform (Sec. 4.3).

A. System Implementation

The system is implemented in Python 3.11+ and exposes both a CLI and an async HTTP API via FastAPI. The CLI provides five primary commands—`clio query`, `clio index`, `clio serve`, `clio list`, and `clio seed`—with flags that control retrieval behavior: `--agentic` and `--max-hops` enable multi-hop retrieval, `--llm-rewrite` activates LLM query reformulation, and `--numeric-range` and `--formula` engage the science-aware operator branches. The HTTP API mirrors the CLI surface and serves as the entry point for programmatic and agent-driven access.

Dense retrieval uses sentence-transformers with the all-MiniLM-L6-v2 model (22.7M parameters, 384-dimensional embeddings, Apache 2.0 license). Two ANN backends are supported behind a common `ANNAdapter` protocol: `ExactANNAdapter`, which partitions vectors across 16 shards and computes exact cosine similarity, and `HnswANNAdapter`, which delegates to hnswlib for approximate search on larger corpora. Write safety on shared filesystems is enforced via `fcntl` file locking. Observability is provided by OpenTelemetry instrumentation with Prometheus-compatible `/metrics` endpoints; all retrieval branches and agentic hops emit structured trace events.

TABLE III
DuckDB storage schema.

Table	Purpose	Key Columns
`documents`	Corpus membership	URI, namespace, SHA-256
`chunks`	Text segments	text, document FK, position
`embeddings`	Dense vectors	384-dim float, chunk FK
`metadata`	Flexible annotations	key-value per doc/chunk
`file_index`	Change detection	path, SHA-256, mtime
`lexical_postings`	BM25 retrieval	token-to-chunk, term freq.
`sci._measurements`	Dimensional search	raw/canonical value+unit
`sci._formulas`	Formula matching	raw expr., normalized form

All persistent state resides in a single DuckDB database file. DuckDB was selected for three reasons: it is embedded (no external service to deploy on compute nodes), it supports SQL-native full-text search sufficient for BM25 scoring, and its single-file storage model is compatible with POSIX parallel filesystems common in HPC environments. The schema comprises eight tables (Table III).

The `scientific_measurements` and `scientific_formulas` tables are the storage-level counterpart of the science-aware operators described in Section 3. At index time, extracted quantities are canonicalized to base SI units and stored with both raw and canonical representations; at query time, the dimensional operator queries canonical columns directly, bypassing the embedding pipeline entirely.

The system is validated by 169 automated tests spanning unit, integration, and benchmark suites, executed via pytest with no external service dependencies.

B. HDF5 and NetCDF Connectors

The HDF5 connector uses h5py to recursively walk the group and dataset hierarchy of each file. For every dataset, the connector extracts the name, shape, dtype, and all attached HDF5 attributes. Attributes carrying physical semantics—`units`, `long_name`, `description`—are identified by convention and routed through the SI canonicalization pipeline described in Section 3.2, producing entries in the `scientific_measurements` table. Remaining attribute text is concatenated into chunk content for hybrid lexical and dense retrieval. Dataset structural metadata (shape, dtype, group path) is stored as key-value pairs in the `metadata` table, enabling queries such as "find all 3D datasets with a pressure variable." Incremental re-indexing uses SHA-256 checksums and filesystem mtime to skip unchanged files.

The NetCDF connector uses xarray and is aware of Climate and Forecast (CF) metadata conventions [42]. For each file, the connector extracts all variables with their names, units, `long_name` attributes, dimensions, and shapes, as well as global attributes including `title`, `history`, and

`Conventions`. Variable-level units pass through the same SI canonicalization pipeline as the HDF5 connector, ensuring that a pressure variable stored in hPa in one NetCDF file and in Pa in another will match the same dimensional query. Global attributes are indexed as document-level metadata. Both connectors share the same base architecture—threading, change detection, lexical ingestion, and scientific extraction—differing only in the format-specific traversal logic.

C. Experimental Platform

All experiments reported in Section 5 were conducted on [TBD — hardware specifications to be added after benchmarking]. The software environment consists of Linux, Python 3.11, DuckDB 1.1, and sentence-transformers 3.3 with the all-MiniLM-L6-v2 embedding model (22.7M parameters, 384 dimensions). The embedding model was selected for its combination of compact size, permissive licensing (Apache 2.0), and demonstrated effectiveness on semantic textual similarity benchmarks [43].

The clio-agentic-search codebase comprises approximately 9,300 lines of Python across 40 modules, with 169 automated tests. The project uses uv for dependency resolution and reproducible environment setup; `uv sync` installs all dependencies from a locked manifest. The full source code, test suite, and evaluation scripts are available as open-source software.

We note two honest limitations of the current prototype relevant to reproducibility. First, the Qdrant and Neo4j connectors use in-memory stub implementations rather than connecting to deployed service instances; they validate the connector protocol and capability negotiation logic but do not exercise network I/O or distributed storage. Second, all experiments in this paper run on a single node. Deploying the federated architecture across multiple HPC sites with heterogeneous storage backends—the scenario motivating the design—remains future work and is discussed in Section 6.

V. Evaluation

We evaluate clio-agentic-search along three research questions. **RQ1**: Does dimensional conversion improve retrieval accuracy over standard baselines on cross-unit queries? **RQ2**: Does formula normalization improve retrieval on formula-variant queries? **RQ3**: Does federated multi-namespace search maintain coverage without sacrificing the gains from science-aware operators? We then present an ablation study isolating the marginal contribution of each pipeline component, analyze multi-hop agentic retrieval behavior, and report indexing throughput.

A. Experimental Setup

Benchmark construction. We construct a controlled benchmark designed to isolate the effect of unit heterogeneity on retrieval. The benchmark spans five SI domains from Table II: distance, mass, time, pressure, and velocity. For each domain, we author document sets in which the same physical quantity is expressed in two or three different units. Each retrieval task consists of a query posed in one unit and a corpus containing relevant documents expressed in different units of the same domain. For example, a query requesting pressures in kPa is paired with documents reporting values in Pa and MPa. The benchmark comprises [TBD] total retrieval tasks. We partition tasks into two conditions: a *control* set in which query and document units match, and a *cross-unit* set in which they differ. The control set establishes that observed differences are attributable to unit mismatch rather than content difficulty.

TABLE IV
Retrieval performance on cross-unit queries (K=5).

Configuration	P@5	R@5	F1@5	MRR
BM25-only	[TBD]	[TBD]	[TBD]	[TBD]
Dense-only	[TBD]	[TBD]	[TBD]	[TBD]
Hybrid BM25+Dense	[TBD]	[TBD]	[TBD]	[TBD]
String normalization	[TBD]	[TBD]	[TBD]	[TBD]
Full pipeline (ours)	[TBD]	[TBD]	[TBD]	[TBD]

Baselines. We compare five retrieval configurations: (1) **BM25-only**, using DuckDB full-text search with default BM25 scoring; (2) **Dense-only**, using all-MiniLM-L6-v2 embeddings (384 dimensions) with exact cosine similarity; (3) **Hybrid BM25+Dense**, combining lexical and vector scores via linear fusion with equal weights; (4) **String normalization**, following the Numbers Matter! approach [8] of normalizing numeric tokens to a canonical string form before indexing; and (5) **Full pipeline**, the complete clio-agentic-search system with dimensional conversion, formula normalization, graph traversal, and agentic rewriting enabled.

Metrics. We report precision at K (P@K), recall at K (R@K), F1 at K (F1@K) for $K \in \{1, 3, 5, 10\}$, and mean reciprocal rank (MRR). We designate K=5 as the primary evaluation point for all comparisons. Statistical significance is assessed via paired bootstrap resampling (10,000 iterations, $p < 0.05$).

B. RQ1: Dimensional Conversion Accuracy

We address the question: *does dimensional conversion improve retrieval accuracy over standard baselines when query and document units differ?*

Table IV reports results on cross-unit queries. The three text-oriented baselines—BM25, dense, and hybrid—retrieve only documents whose surface text contains the same unit string as the query. A query for "200 kPa" does not lexically match a document containing "200000 Pa," and the embedding model has no mechanism to learn that these denote identical physical quantities. Consequently, these baselines are limited to retrieving same-unit documents that happen to appear in the cross-unit corpus, resulting in P@5, R@5, and MRR scores of [TBD], [TBD], and [TBD] respectively for the strongest text baseline (Hybrid).

The string normalization baseline partially addresses this gap by canonicalizing numeric tokens before indexing. This strategy can match values when unit abbreviations coincide or when simple lexical rewriting rules apply, yielding R@5 of [TBD]. However, string normalization cannot perform the

arithmetic conversion required to recognize that 200 kPa equals 200000 Pa—it operates on token forms, not physical semantics. Our full pipeline bypasses this limitation entirely: at index time, all measurements are converted to canonical SI base units and stored in the `scientific_measurements` table; at query time, the dimensional operator converts the query bounds to the same canonical space and issues a direct range predicate. This yields R@5 of [TBD] and MRR of [TBD] on cross-unit queries, representing an absolute improvement of [TBD] points in R@5 over the next-best baseline.

On the same-unit control set, all five configurations achieve comparable performance (P@5 within [TBD] points of each other), confirming that the observed gains are attributable specifically to cross-unit matching rather than general retrieval quality differences.

Per-domain breakdown. We disaggregate results across the five SI domains. Pressure and velocity queries, which involve conversion factors spanning three orders of magnitude, show the largest baseline degradation and the largest improvement from dimensional conversion. Distance queries exhibit intermediate gains, while mass and time queries—where unit variation in our benchmark is less extreme—show smaller but still statistically significant improvements ([TBD] across all domains, $p < 0.05$). Fig. 4 presents R@5 across all five baselines, grouped by SI domain.

C. RQ2: Formula Matching Accuracy

We address the question: *does formula normalization improve retrieval accuracy on queries involving mathematical notation variants?*

We construct a set of formula-variant retrieval tasks in which a query references a formula in one notation and relevant documents express the same relationship in alternative forms. Variants include reordering of terms (e.g., $F = ma$ vs. $F = am$), equivalent symbolic rearrangements (e.g., $E = mc^2$ vs. $m = E/c^2$), and differing variable naming conventions. Each task has a known set of relevant documents.

Without formula operators, the system relies on lexical and embedding similarity over raw LaTeX or Unicode strings, achieving P@5 of [TBD] and R@5 of [TBD]. Enabling formula normalization—which canonicalizes expressions into a sorted, simplified symbolic form before indexing and matching—raises P@5 to [TBD] and R@5 to [TBD]. The improvement is statistically significant ($p < 0.05$) and consistent across algebraic, differential, and integral formula classes.

We also measure the interaction between formula and dimensional operators. Enabling both operators jointly yields F1@5 of [TBD], compared to [TBD] for formula-only and [TBD] for dimensional-only, indicating that the two operators provide independent and complementary value. Neither operator's contribution subsumes the other, validating the pluggable design described in Section 3.

D. RQ3: Federated Search Coverage

We address the question: *does federated multi-namespace search maintain retrieval coverage when the same corpus is distributed across heterogeneous backends?*

TABLE V
ABLATION STUDY ON CROSS-UNIT QUERIES.

Config.	Components	F1@5
A: Lexical	BM25	[TBD]
B: +Vector	A + dense embeddings	[TBD]
C: +Scientific	B + dimensional + formula ops.	[TBD]
D: +Graph	C + entity-rel. traversal	[TBD]
E: +Agentic	D + multi-hop query rewriting	[TBD]
F: Full	E + all optimizations	[TBD]

We replicate the benchmark corpus across three storage backends: local filesystem, S3-compatible object store (MinIO), and HDF5 files. Each backend is registered as a separate namespace. We measure coverage—the fraction of relevant documents retrieved regardless of source backend—under single-backend and federated configurations, both with and without science-aware operators.

Single-backend search restricted to the filesystem namespace retrieves [TBD]% of relevant documents. Restricting to S3 or HDF5 alone yields [TBD]% and [TBD]% respectively, reflecting the partitioned placement of corpus documents. Federated search across all three namespaces achieves [TBD]% coverage, recovering documents that any single backend misses.

Critically, science-aware operators compose with federation without degradation. Federated search with dimensional conversion achieves R@5 of [TBD] on cross-unit queries, compared to [TBD] for federated search without science operators. The capability negotiation mechanism (Section 3.5) ensures that connectors lacking scientific measurement tables are gracefully skipped rather than producing errors, so adding a backend that does not support dimensional search does not degrade results from backends that do.

E. Ablation Study

To isolate the marginal contribution of each pipeline component, we evaluate six cumulative configurations on the cross-unit query set:

Fig. 5 plots F1@5 for each configuration. The transition from B to C—adding scientific operators—produces the largest marginal gain ([TBD] absolute F1@5 improvement), confirming that cross-unit matching is the dominant bottleneck for this query class. Adding vector retrieval to the lexical baseline (A to B) provides moderate improvement ([TBD] points), consistent with the known complementarity of sparse and dense retrieval. Graph traversal (C to D) contributes [TBD] points, primarily on queries requiring entity disambiguation. Agentic rewriting (D to E) yields [TBD] additional points, with gains concentrated on multi-hop queries where the initial query formulation is underspecified.

On the same-unit control queries, the A-through-F progression is monotonically non-decreasing but the scientific operator transition (B to C) provides negligible gain ([TBD]

points), confirming that this component's value is specific to the cross-unit retrieval scenario it was designed for.

F. Multi-Hop Agentic Retrieval

We evaluate the agentic retrieval loop (Section 3.6) on [TBD] complex queries requiring information synthesis across multiple documents. We compare single-pass retrieval (1 hop), two-hop, and three-hop configurations.

Single-pass retrieval achieves R@5 of [TBD]. Two-hop retrieval improves to [TBD], with the second hop refining the query based on entities and measurements extracted from first-hop results. Three-hop retrieval yields [TBD], a marginal improvement over two hops, suggesting that for most queries in our benchmark the agentic loop converges within two iterations.

We illustrate the rewriting behavior with representative examples. Consider the query "What simulation used pressures around 200 kPa with a velocity of 50 km/h?" In the first hop, the system retrieves documents matching the dimensional constraints. The agentic rewriter examines these results, identifies a referenced simulation identifier, and reformulates the query for the second hop to include the identifier as a lexical constraint, recovering additional documents linked by that identifier but not by the original numeric constraints.

For deployments where LLM inference is unavailable or latency-constrained, the system falls back to a deterministic rewriting strategy: the rewriter extracts entities and measurement tuples from first-hop results and appends them as structured filters to the second-hop query. On the same benchmark, the non-LLM fallback achieves R@5 of [TBD], compared to [TBD] for LLM-based rewriting—a modest degradation that preserves the majority of multi-hop gains without requiring a language model at query time.

G. Indexing Performance

We measure indexing throughput on a single compute node (Section 4.3) across the three connector types.

Full indexing of the benchmark corpus ([TBD] documents) completes in [TBD] seconds, corresponding to [TBD] documents/second. Per-connector throughput varies with document complexity: the filesystem connector processes [TBD] documents/second, S3 achieves [TBD] documents/second (dominated by network round-trip latency), and the HDF5 connector processes [TBD] files/second, with per-file time scaling with dataset count and attribute density.

Incremental re-indexing, triggered by SHA-256 checksum changes in the `file_index` table, substantially reduces update cost. Fig. 6 plots indexing time as a function of corpus change fraction. When 5% of documents are modified, incremental indexing completes in [TBD]% of the full-index time. At 10% and 25% change fractions, incremental indexing requires [TBD]% and [TBD]% of full-index time respectively. The sublinear scaling arises because unchanged documents are skipped entirely—no text extraction, embedding computation, or measurement canonicalization is performed—and only the modified chunks and their dependent entries in the `embeddings`, `scientific_measurements`, and `scientific_formulas` tables are rewritten.

VI. Conclusion

Scientific data retrieval at HPC facilities fails not because retrieval algorithms are weak, but because they are blind to the physical semantics of the data they index. When a scientist asks for pressures between 190 and 360 kPa, no existing system recognizes that "200000 Pa" in an HDF5 attribute and "0.2 MPa" in an S3-hosted dataset both satisfy the query. We addressed this gap with clio-agentic-search, a retrieval system built around science-aware operators—deterministic, pluggable primitives that perform arithmetic SI conversion, formula normalization, and federated multi-backend search as first-class retrieval branches alongside standard BM25 and dense vector pipelines.

The results validate the core hypothesis. Dimensional-conversion operators recover [TBD]% of cross-unit false negatives that lexical, dense, and string-normalization baselines miss entirely, achieving R@5 of [TBD] on queries where the strongest text baseline scores [TBD]. Formula normalization adds [TBD] F1 on notation-variant queries, and the two operators compose independently—neither subsumes the other. Federated search across filesystem, S3, and HDF5 backends discovers [TBD]% more relevant documents than any single-backend configuration, while capability negotiation ensures that adding a backend without scientific measurement support does not degrade results from backends that have it. The multi-hop agentic loop converges within 2–3 hops, improving recall by [TBD] over single-pass retrieval. These gains matter because AI agents are now orchestrating scientific workflows end-to-end, from hypothesis generation through experiment execution, yet their retrieval layers treat measurements as opaque strings.

Our current prototype covers 13 units across 5 SI domains, far fewer than the 531 units in CQE [27] or the thousands in the QUDT ontology; derived and compound units remain unsupported. Measurement extraction relies on regular expressions rather than dependency parsing, which limits robustness on syntactically complex scientific prose. The composite fusion score uses hand-tuned weights rather than learned reranking via cross-encoders or late-interaction models. All experiments run on a single-node benchmark corpus, not a petabyte-scale parallel filesystem—the federated architecture is designed for scale, but we have not yet validated it at ALCF or OLCF production workloads.

Several directions follow naturally. Integrating the QUDT ontology would extend unit coverage to derived and compound units without architectural changes. Wrapping the retrieval API as a Model Context Protocol server [4] would expose science-aware operators to any MCP-compatible agent framework. Deployment on DOE leadership-class systems—Polaris, Frontier, Aurora—will test federated search at the scale it was designed for. More broadly, the principle that domain knowledge should execute as verifiable arithmetic within the retrieval pipeline,

rather than be approximated by learned representations, extends beyond SI units to chemical identifiers, genomic coordinates, and any domain where correctness is non-negotiable. If retrieval systems are to serve as the foundation for autonomous scientific agents, they must understand the data they search—not merely the text that describes it.

References

[1] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, pp. 570–578, 2023.

[2] Y. Yamada, R. T. Lange, C. Guo, K. Irie, Y. Tang, and D. Ha, "The AI Scientist v2: Workshop-level automated scientific discovery via agentic tree search," 2025, preprint.

[3] Oak Ridge National Laboratory, "Autonomous agents for experiment orchestration across national computing facilities," in *SC'25 Workshops*, 2025.

[4] Model Context Protocol for HPC, "Extending the model context protocol for Globus search, compute, and storage in HPC," 2025, arXiv:2508.18489.

[5] Z. Chen, S. Chen, Y. Ning, Q. Zhang, B. Wang, B. Yu, Y. Li, Z. Liao, C. Wei, Z. Lu, V. Dey, M. Xue, F. N. Baker, B. Burns, D. Adu-Ampratwum, X. Huang, X. Ning, S. Gao, Y. Su, and H. Sun, "ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025, arXiv:2410.05080.

[6] Q. Huang, J. Vora, P. Liang, and J. Leskovec, "MLAgentBench: Evaluating language agents on machine learning experimentation," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.

[7] Agentic AI Survey Authors, "A survey of agentic AI: From reasoning to tool use and beyond," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[8] S. Almasian, D. Aumiller, and M. Gertz, "Numbers matter! bringing quantity-awareness to retrieval systems," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, arXiv:2407.10283.

[9] S. Shrestha *et al.*, "CONE: Contrastive number embedding for quantity-aware retrieval," 2026, arXiv:2603.04741.

[10] K. Deng *et al.*, "The numeracy gap: Evaluating embedding models on numerical content retrieval," in *Proceedings of the European Chapter of the Association for Computational Linguistics (EACL)*, 2026, arXiv:2509.05691.

[11] NC-Retriever Authors, "NC-Retriever: Numeric constraint-aware dense retrieval," 2025.

[12] Approach0, "Approach0: A math-aware search engine," https://approach0.xyz/, accessed 2025.

[13] SSEmb Authors, "SSEmb: Semantic similarity embeddings for mathematical expressions," in *Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM)*, 2025.

[14] R. Guerraoui *et al.*, "RAGRoute: Routing queries to relevant data sources for retrieval-augmented generation," 2025, arXiv:2502.19280.

[15] PROV-IO+ Authors, "PROV-IO+: A cross-platform provenance framework for scientific data with minimal overhead," *IEEE Transactions on Parallel and Distributed Systems*, 2024.

[16] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[17] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[18] S. Bruch, S. Gai, and A. Ingber, "An analysis of fusion functions for hybrid retrieval," *ACM Transactions on Information Systems*, vol. 42, no. 1, 2023.

[19] T. Formal, B. Piwowarski, and S. Clinchant, "SPLADE: Sparse lexical and expansion model for first stage ranking," in *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2021.

[20] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[21] Z. Du *et al.*, "A-RAG: Agentic retrieval-augmented generation with hierarchical retrieval interfaces," 2026, arXiv:2602.03442.

[22] Numbers Not Continuous Authors, "Why numbers break neural retrieval: The discontinuous representation problem," 2025, arXiv:2510.08009.

[23] NumericBench Authors, "NumericBench: A benchmark for numerical reasoning in language models," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

[24] Number Cookbook Authors, "The number cookbook: A systematic catalog of numerical failure modes in large language models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[25] Apple, "GSM-Symbolic: Understanding the limitations of mathematical reasoning in large language models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[26] S. Almasian and M. Gertz, "QFinder: Quantity-aware search in elasticsearch," in *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2022.

[27] S. Almasian, D. Aumiller, and M. Gertz, "CQE: Complex quantity extraction from scientific text," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, arXiv:2305.08853.

[28] O. Weller *et al.*, "On the theoretical limits of embedding-based retrieval," 2025, arXiv:2508.21038.

[29] Y. Ju and Y. Dong, "MIRB: A comprehensive benchmark for math information retrieval," 2025, arXiv:2505.15585.

[30] A. Asai *et al.*, "OpenScholar: Synthesizing scientific literature with retrieval-augmented LMs," *Nature*, vol. 650, 2026.

[31] HiPerRAG Authors, "HiPerRAG: Scaling retrieval-augmented generation to millions of scientific articles on DOE supercomputers," in *Platform for Advanced Scientific Computing (PASC)*, 2025, arXiv:2505.04846.

[32] PANGAEA-GPT Authors, "PANGAEA-GPT: Hierarchical multi-agent search over geoscientific datasets," 2026, arXiv:2602.21351.

[33] Y. Ning *et al.*, "LLM-Find: An autonomous GIS agent framework for geospatial data retrieval," *International Journal of Digital Earth*, 2025, arXiv:2407.21024.

[34] Idaho National Laboratory, "Datum: A scientific data catalog for INL," 2024.

[35] BRINDEXER Authors, "BRINDEXER: Balanced-range indexer for POSIX metadata on Lustre," in *Proceedings of the IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2020.

[36] HDF Clinic Authors, "HDF Clinic: A knowledge graph approach to HDF5 data management," 2025.

[37] A. Mishra *et al.*, "SoK: Agentic retrieval-augmented generation — a systematization of knowledge," 2026, arXiv:2603.07379.

[38] Chroma, "Context-1: 20-billion-token agentic search," 2026.

[39] CLADD Authors, "CLADD: Collaborative agent-based drug discovery with domain knowledge graphs," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025, arXiv:2502.17506.

[40] SciAgent Authors, "SciAgent: Hierarchical multi-agent coordination for scientific workflows," 2025, arXiv:2511.08151.

[41] M. J. Buehler *et al.*, "ScienceClaw: A 300+ skill agent swarm with immutable artifact provenance," 2026, arXiv:2603.14312.

[42] CF Conventions Committee, "CF metadata conventions for NetCDF," https://cfconventions.org/, accessed 2025.

[43] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.